Practical No:1

Aim: Design a simple linear neural network model

WRITE-UP – A) Design a simple linear neural network model

Aim: To design a simple linear neural network model and understand how input data is processed to produce an output.

Theory: A neural network consists of interconnected processing units. The model receives input sequences, processes them through a neural layer and produces an output using an activation function.

Result: The simple neural network model is designed and trained successfully.

Source Code:

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import SimpleRNN,Dense, Input # Import Input layer
import numpy as np # Import numpy
text="This is GeeksforGeeks a software training institute"
chars=sorted(list(set(text)))
char_to_index={char:i for i, char in enumerate(chars)} # Fixed variable name
index_to_chars={i:char for i, char in enumerate(chars)} # Fixed variable name
seq_length =3
sequences=[] # Fixed typo 'sequqnces'
labels=[]
for i in range(len(text)-seq_length):
  seq=text[i:i+seq_length] # Fixed typo 'tex'
  label=text[i+seq_length] # Corrected assignment for label
  sequences.append([char_to_index[char] for char in seq])
  labels.append(char_to_index[text[i+seq_length]])
x=np.array(sequences)
y=np.array(labels)
X_one_hot=tf.one_hot(x,len(chars)) # Fixed variable name
Y_one_hot=tf.one_hot(y,len(chars)) # Fixed variable name
model = Sequential()
# Add an Input layer explicitly to resolve the UserWarning
model.add(Input(shape=(seq_length, len(chars))))
model.add(SimpleRNN(50, activation ='relu')) # Removed
```

input_shape as it's now handled by Input layer
model.add(Dense(len(chars), activation='softmax'))
model.compile(optimizer='adam',loss
='categorical_crossentropy',metrics=['accuracy'])
model.fit(X_one_hot,Y_one_hot,epochs=100)

OUTPUT:

```
Epoch 1/100
2/2 ━━━━━━━━━━━━━━━━━━━━ 1s 25ms/step - accuracy: 0.0417 - loss: 2.8307
Epoch 2/100
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - accuracy: 0.0625 - loss: 2.8042
Epoch 3/100
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - accuracy: 0.1042 - loss: 2.7804
Epoch 4/100
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - accuracy: 0.1458 - loss: 2.7575
Epoch 5/100
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - accuracy: 0.1458 - loss: 2.7367
Epoch 6/100
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.1875 - loss: 2.7141
Epoch 7/100
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - accuracy: 0.1875 - loss: 2.6934
Epoch 8/100
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - accuracy: 0.1875 - loss: 2.6720
Epoch 9/100
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - accuracy: 0.2292 - loss: 2.6502
Epoch 10/100
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - accuracy: 0.2917 - loss: 2.6283
Epoch 11/100
```

B) **Calculate the output of neural network using both binary & bipolar sigmoidal function**

WRITE-UP – B) Binary and bipolar sigmoidal function

Aim: To calculate the output of a neural network using binary and bipolar sigmoid functions.

Theory: Sigmoid functions convert the net input of a neuron into an output value. Binary sigmoid gives values in the positive range, while bipolar sigmoid represents output around positive and negative values.

Result: The neural network output is calculated using the specified sigmoid functions.

CODE:

```
start_seq="This is G"
generated_text=start_seq
for i in range(50):
  x=np.array([[char_to_index[char]for char in generated_text[-seq_length:]]])
  X_one_hot=tf.one_hot(x,len(chars))
  prediction=model.predict(X_one_hot)
  next_index=np.argmax(prediction)
```

```
next_char=index_to_chars[next_index]
generated_text+=next_char
print("Generated Text:")
print(generated_text
```

OUTPUT:

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 146ms/step
Generated Text:
This is Ge
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 97ms/step
Generated Text:
This is Gee
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 60ms/step
Generated Text:
This is Geek
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 139ms/step
Generated Text:
This is Geeks
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step
Generated Text:
This is Geeksf
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step
Generated Text:
This is Geeksfo
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 35ms/step
Generated Text:
This is Geeksfor
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 38ms/step
Generated Text:
```

PRACTICAL: 2

WRITE-UP – A) AND/OR using McCulloch-Pitts neuron

Aim: To implement the AND/OR logical function using a McCulloch-Pitts neuron.

Theory: The McCulloch-Pitts neuron is a threshold-based neuron. It calculates a weighted sum of binary inputs and produces an output when the threshold condition is satisfied.

Result: The required logical output is generated for the given binary inputs.

A) Implement AND/OR Function using McCulloch-pitts neuron (Use binary data and in presentation)

CODE:

```
num_ip=int(input("Enter the number of inputs: "))
#Step 2:Weights
w1=1
 w2=2
 print(f"For the{num_ip}inputs,calculate the net input using
Yin=x1*w1+x2*w2")
#Step 3:Collect inputs
x1=[]
x2=[]
for j inrange(num_ip):
ele1 =int(input(f"x1[{j+1}]="))
ele2 =int(input(f"x2[{j+1}]="))
x1.append(ele1)
 x2.append(ele2)
 print("x1=",x1)
 print("x2=",x2)
 #Step 4: Weighted inputs(elementwise multiplication)
 n=[val * w1 forvalin x1] m=[val*w2 forvalin x2]
 #Step 5: Net input calculation(excitatory weights)
 Yin=[n[i]+m[i]foriinrange(num_ip)]
 print("Yin(excitatory)=", Yin)
 #Step 6: Net input calculation(one excitatory, one inhibitory)
 Yin_inhib=[n[i]-m[i]
for print("Yin(one inhibitory)="
#Step 7: Activation function(threshold at 1)
 Y=[] foriinrange(num_ip):
 If Yin_inhib[i]>=1:
 Y.append(1)
 else:
 Y.append(0)
 print("Final Output Y="
```

OUTPUT:

```
Enter the number of inputs: 2
For the2inputs,calculate the net input using Yin=x1*w1+x2*w2
x1[1]=1
x2[1]=3
x1[2]=2
x2[2]=4
x1= [1, 2]
x2= [3, 4]
Yin(excitatory)= [7, 10]
Yin(one inhibitory)= [-5, -6]
Final Output Y= [0, 0]
```

WRITE-UP – B) XOR using McCulloch-Pitts neural network

Aim: To generate the XOR function using a McCulloch-Pitts neural network.

Theory: XOR is not linearly separable, so a network with hidden neurons is used to combine the required logical conditions and generate the final output.

Result: The network generates the XOR output for the given input combinations.

B) **Generate XOR function using McCulloch-pitts neuron network.**

CODE:

```
import numpy as np
print("Enter the Weight")
w11=int(input("Weight w11="))
w12=int(input("Weight w12="))
w21=int(input("Weight w21="))
w22=int(input("Weight w22= "))
v1=int(input("Weight v1="))
v2=int(input("Weight v2="))
print("Enter the threshold value")
theta=int(input("theta="))
 #XOR trurh table inputs and outputs
x1=np.array([0,0,1,1])
x2=np.array([0,1,0,1])
z=np.array([0,1,1,0])
 con=1
y1=np.zeros((4,))
y2=np.zeros((4,))
y=np.zeros((4,))
 while con==1:
#Hidden layer weighted sums
  zin1=x1*w11+x2*w21
  zin2=x1*w12+x2*w22 #Fixed:use w12 here instead of w21
  print("z1=",zin1)
  print("z2=",zin2)
 #Apply threshold to hidden layer neurons
  for i in range(4):
   y1[i]=1 if zin1 [i]>=theta else 0
```

```
    y2[i]=1 if zin2[i]>=theta else 0
  #output neuron calculation
  yin =y1*v1+y2*v2
  for i in range(4):
    y[i]=1 if yin[i]>=theta else 0
  print("yin=",yin)
  print("Output of Net")
  y=y.astype(int)
  print("y=",y)
  print("z=",z)
   #Check if output matches XOR
  if np.array_equal(y,z):
    con=0
  else:
    print("Net is not learning, enter another set of weights and threshold
value")
    w11=int(input("Weight w11="))
    w12=int(input("Weight w12="))
    w21=int(input("Weight w21= "))
    w22=int(input("Weight w22= "))
    v1=int(input("Weight v1="))
    v2=int(input("Weight v2="))
    theta=int(input("theta="))
  print("McCulloch-Pits Net for XOR function")
  print("Weights of Neuron Z1:w11=",w11,"w21=",w21)
  print("Weights of Neuron Z2:w12=",w12,"w22=",w22)
  print("Weights of Neuron Y:v1=",v1,"v2=",v2)
  print("Threshold value:",theta)
```

OUTPUT:

```
Weight w11=1
Weight w12=-1
Weight w21= -1
Weight w22= 1
Weight v1=1
Weight v2=1
theta=1
McCulloch-Pits Net for XOR function
Weights of Neuron Z1:w11= 1 w21= -1
Weights of Neuron Z2:w12= -1 w22= 1
Weights of Neuron Y:v1= 1 v2= 1
Threshold value: 1
z1= [ 0 -1  1  0]
z2= [ 0  1 -1  0]
yin= [0. 1. 1. 0.]
Output of Net
y= [0 1 1 0]
z= [0 1 1 0]
McCulloch-Pits Net for XOR function
Weights of Neuron Z1:w11= 1 w21= -1
Weights of Neuron Z2:w12= -1 w22= 1
Weights of Neuron Y:v1= 1 v2= 1
Threshold value: 1
```

PRACTICAL: 3

WRITE-UP – A) Hebb's Rule

Aim: To implement Hebb's learning rule.

Theory: Hebbian learning updates connection weights according to the input and neuron response. Connections between simultaneously active neurons become stronger.

Result: New weight values are calculated using Hebb's learning rule.

A) Implement Hebb's Rule

CODE:

```
x1= float(input("enter value for x1:"))
x2= float(input("enter value for x2:"))
w1= float(input("enter value for w1:"))
w2= float(input("enter value for w2:"))
yin= float(1+(x1*w1+x2*w2))
print("value of yin is:", yin)
y= float(1/1+(2.178*yin))
print("value of y is:",y)
w3= float(y*x1)
w4= float(y*x2)
print("value of 1 new weigth is :",w3)
print("value of 2 new weight is:", w4)
ynew= float(1+(x1*w1+x2*w4))
print("new value of y is :",ynew)
```

OUTPUT:

```
enter value for x1:3
enter value for x2:2
enter value for w1:4
enter value for w2:1
value of yin is: 15.0
value of y is: 33.67
value of 1 new weigth is : 101.01
value of 2 new weight is: 67.34
new value of y is : 147.68
```

WRITE-UP – B) Delta Learning Rule

Aim: To implement the Delta learning rule.

Theory: The Delta rule is a supervised learning method in which weights are adjusted using the difference between desired and actual output.

Result: The learning calculation is performed for the supplied input values.

3b) Implement Delta Learning rule.

CODE:

```
x1= float(input("enter value of x1:"))
x2= float(input("enter value of x2:"))
```

```
e= float(input("enter value of e:"))
eq= float(x1*x2)
print("value of y:", eq)
if (eq>=1):
  print("learning is supervised")
else:
  print("learning is unsupervised")
```

OUTPUT:

```
enter valuue of x1:3
enter value of x2:1
enter value of e:3
value of y: 3.0
learning is supervised
enter valuue of x1:0
enter value of x2:2
enter value of e:1
value of y: 0.0
learning is unsupervised
```

PRACTICAL: 4

WRITE-UP – A) Back Propagation Algorithm

Aim: To implement the backpropagation algorithm.

Theory: Backpropagation is a supervised learning technique. The network performs forward propagation, calculates output error and propagates the error backward to update weights.

Result: The network learns by repeatedly updating its parameters and produces a learned response.

A) Write a program for backpropagation algorithm Source Code

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap

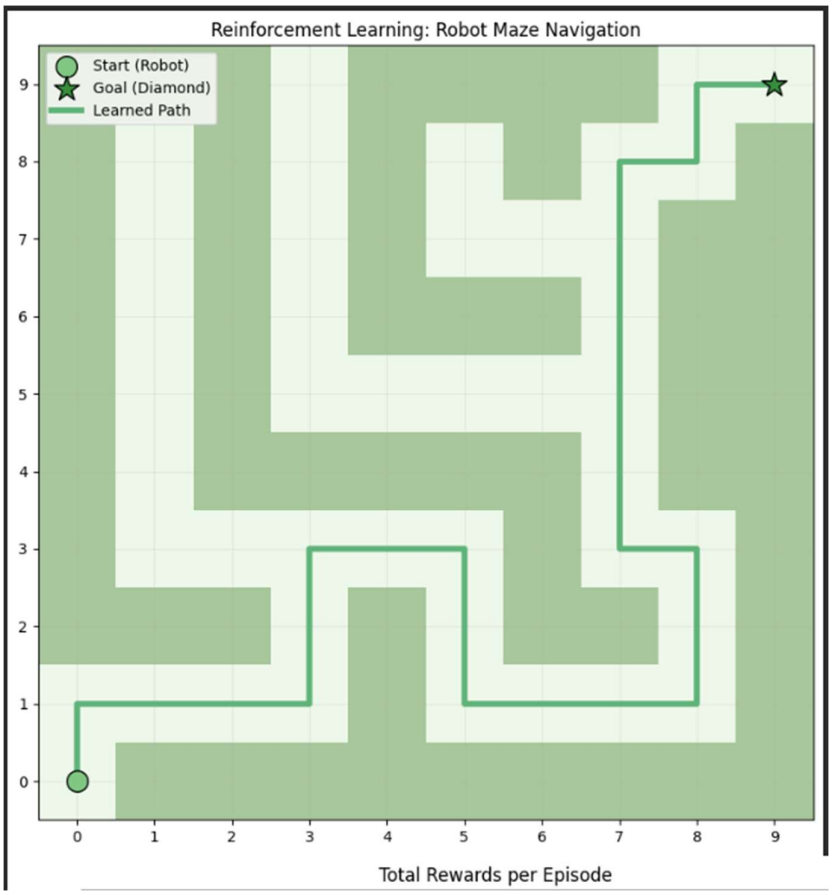
maze = np.array([
   [0, 1, 1, 1, 1, 1, 1, 1, 1, 1],
   [0, 0, 0, 0, 1, 0, 0, 0, 0, 1],
   [1, 1, 1, 0, 1, 0, 1, 1, 0, 1],
   [1, 0, 0, 0, 0, 0, 1, 0, 0, 1],
   [1, 0, 1, 1, 1, 1, 1, 0, 1, 1],
   [1, 0, 1, 0, 0, 0, 0, 0, 1, 1],
   [1, 0, 1, 0, 1, 1, 1, 0, 1, 1],
   [1, 0, 1, 0, 1, 0, 0, 0, 1, 1],
   [1, 0, 1, 0, 1, 0, 1, 0, 0, 1],
   [1, 1, 1, 0, 1, 1, 1, 1, 0, 0]
])
start = (0, 0)
goal = (9, 9)
num_episodes = 5000
alpha = 0.1
gamma = 0.9
epsilon = 0.5
reward_fire = -10
reward_goal = 50
reward_step = -1
actions = [(0, -1), (0, 1), (-1, 0), (1, 0)]
Q = np.zeros(maze.shape + (len(actions),))
def is_valid(pos):
  r, c = pos
  if r < 0 or r >= maze.shape[0]:
    return False
```

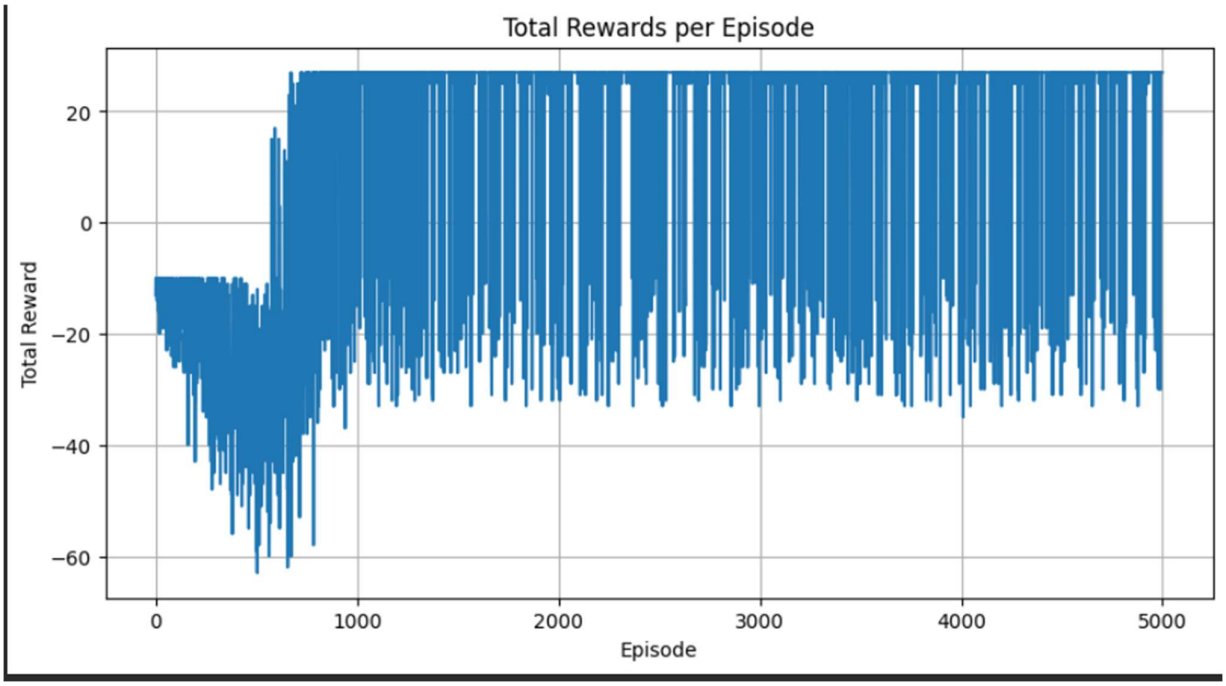
```
  if c < 0 or c >= maze.shape[1]:
    return False
  if maze[r, c] == 1:
    return False
  return True
def choose_action(state):
  if np.random.random() < epsilon:
    return np.random.randint(len(actions))
  else:
    return np.argmax(Q[state])
# Train the Agent with Q-Learning Algorithm
rewards_all_episodes = []
for episode in range(num_episodes):
    state = start
    total_rewards = 0
    done = False
    while not done:
        action_index = choose_action(state)
        action = actions[action_index]
        next_state = (state[0] + action[0], state[1] + action[1])
        if not is_valid(next_state):
            reward = reward_fire
            done = True
        elif next_state == goal:
            reward = reward_goal
            done = True
        else:
            reward = reward_step
        old_value = Q[state][action_index]
        next_max = np.max(Q[next_state]) if is_valid(next_state) else 0
        Q[state][action_index] = old_value + alpha * (reward + gamma
* next_max - old_value)
        state = next_state
        total_rewards += reward
    epsilon = max(0.01, epsilon * 0.995)
    rewards_all_episodes.append(total_rewards)
```

```
# Extract the Optimal Path after Training
def get_optimal_path(Q, start, goal, actions, maze, max_steps=200):
    path = [start]
    state = start
    visited = set()
    for _ in range(max_steps):
        if state == goal:
            break
        visited.add(state)
        best_action = None
        best_value = -float('inf')
        for idx, move in enumerate(actions):
            next_state = (state[0] + move[0], state[1] + move[1])
            if (0 <= next_state[0] < maze.shape[0] and
                0 <= next_state[1] < maze.shape[1] and
                maze[next_state] == 0 and
                next_state not in visited):
                if Q[state][idx] > best_value:
                    best_value = Q[state][idx]
                    best_action = idx
        if best_action is None:
            break
        move = actions[best_action]
        state = (state[0] + move[0], state[1] + move[1])
        path.append(state)
    return path
optimal_path = get_optimal_path(Q, start, goal, actions, maze)
# Visualize the Maze, Robot Path, Start and Goal
def plot_maze_with_path(path):
    cmap = ListedColormap(['#eef8ea', '#a8c79c'])
    plt.figure(figsize=(8, 8))
    plt.imshow(maze, cmap=cmap)
        plt.scatter(start[1], start[0], marker='o', color='#81c784', edgecolors='black',
            s=200, label='Start (Robot)', zorder=5)
```

```
        plt.scatter(goal[1], goal[0], marker='*', color='#388e3c',
edgecolors='black',
            s=300, label='Goal (Diamond)', zorder=5)
    rows, cols = zip(*path)
            plt.plot(cols, rows, color='#60b37a', linewidth=4,
label='Learned Path', zorder=4)
   plt.title('Reinforcement Learning: Robot Maze Navigation')
    plt.gca().invert_yaxis()
    plt.xticks(range(maze.shape[1]))
    plt.yticks(range(maze.shape[0]))
    plt.grid(True, alpha=0.2)
    plt.legend()
    plt.tight_layout()
    plt.show()
plot_maze_with_path(optimal_path)
# Plot Rewards per Training
def plot_rewards(rewards):
    plt.figure(figsize=(10, 5))
    plt.plot(rewards)
    plt.title('Total Rewards per Episode')
    plt.xlabel('Episode')
    plt.ylabel('Total Reward')
    plt.grid(True)
    plt.show()
plot_rewards(rewards_all_episodes)
```

OUTPUT:


Reinforcement Learning: Robot Maze Navigation
Start (Robot)
Goal (Diamond)
Learned Path
0
1
2
3
4
5
6
7
8
9
Total Rewards per Episode


Total Rewards per Episode
Total Reward
20
0
−20
−40
−60
0
1000
2000
3000
4000
5000
Episode

B) Error Backpropagation Algorithm

WRITE-UP – B) Error Backpropagation

Aim: To implement the error backpropagation algorithm.

Theory: Error backpropagation minimizes the difference between target and predicted outputs by calculating the error and distributing it backward through the network for weight adjustment.

Result: The error is used to update the network weights during learning.

Source Code-

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn import datasets
from sklearn.semi_supervised import LabelPropagation
from sklearn.metrics import accuracy_score

iris = datasets.load_iris()
X = iris.data[:, :2]
y = iris.target

labels = np.copy(y)
rng = np.random.RandomState(42)
mask = rng.rand(len(y)) < 0.1
labels[~mask] = -1
print(f"Labeled: {np.sum(mask)}, Unlabeled: {np.sum(~mask)}")

model = LabelPropagation()
model.fit(X, labels)
y_pred = model.transduction_
acc_labeled = accuracy_score(y[mask], y_pred[mask])
acc_overall = accuracy_score(y, y_pred)
print(f"Acc (on original labeled subset): {acc_labeled:.3f}")
print(f"Acc (overall after propagation): {acc_overall:.3f}")

fig, ax = plt.subplots(1, 2, figsize=(12, 4))

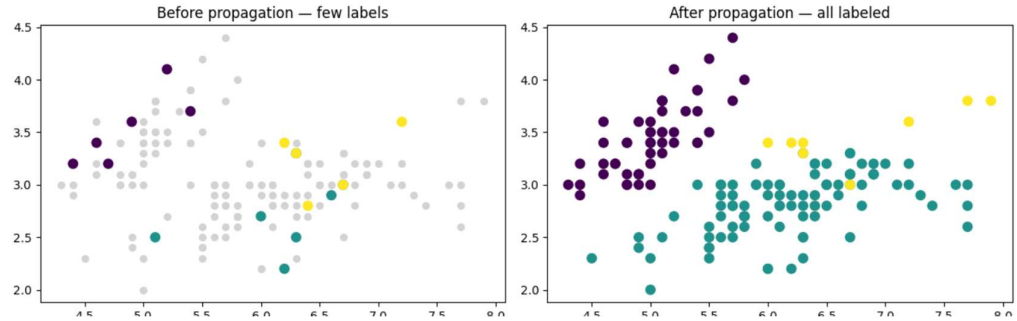
ax[0].scatter(X[:, 0], X[:, 1], c='lightgray', s=30)
ax[0].scatter(X[mask, 0], X[mask, 1], c=y[mask], cmap='viridis', s=60)
ax[0].set_title("Before propagation — few labels")

ax[1].scatter(X[:, 0], X[:, 1], c=y_pred, cmap='viridis', s=60)
ax[1].set_title("After propagation — all labeled")

plt.tight_layout()
plt.show()
```

OUTPUT:

```
Labeled: 18, Unlabeled: 132
Acc (on original labeled subset): 1.000
Acc (overall after propagation): 0.707
```



PRACTICAL: 5

A) Hopfield Network

Aim: To implement a Hopfield network.

Theory: A Hopfield network is a recurrent neural network used for associative memory. Patterns are stored as stable states and the network can converge to a stored pattern from a related input.

Result: The Hopfield network stores and recalls patterns as associative memories.

SOURCE CODE:

```
import numpy as np

patterns = np.array([
  [1, -1, 1, -1],
  [-1, 1, -1, 1]
])

n = patterns.shape[1]
weights = np.zeros((n, n))

for p in patterns:
  weights += np.outer(p, p)

np.fill_diagonal(weights, 0)
weights = weights / n

print("Weight Matrix:")
print(weights)

input_pattern = np.array([1, -1, -1, -1])
output = input_pattern.copy()

for _ in range(10):
  for i in range(n):
    net = np.dot(weights[i], output)
    output[i] = 1 if net >= 0 else -1

print("Input Pattern:", input_pattern)
print("Recalled Pattern:", output)
```

OUTPUT:

```
Weight Matrix:
[[ 0.  -0.5  0.5 -0.5]
 [-0.5  0.  -0.5  0.5]
 [ 0.5 -0.5  0.  -0.5]
 [-0.5  0.5 -0.5  0. ]]
Input Pattern: [ 1 -1 -1 -1]
Recalled Pattern: [ 1 -1  1 -1]
```

PRACTICAL: 7

WRITE-UP – A) Linear Separation

Aim: To implement linear separation.

Theory: Linear separation determines whether different classes of points can be divided by a straight decision boundary. Points on different sides of a suitable line belong to different classes.

Result: Candidate separating lines are plotted and the separation of the given points is demonstrated.

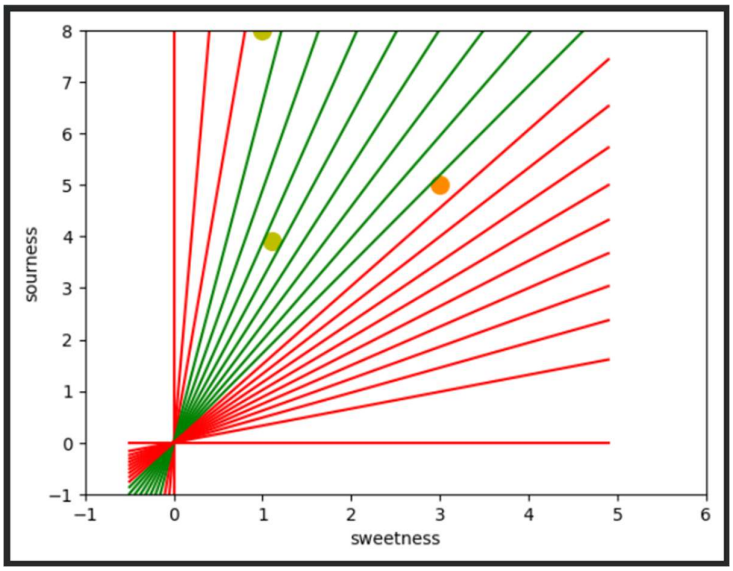
A) Linear separation

Source Code:

```
import numpy as np
import matplotlib.pyplot as plt
def create_distance_function(a, b, c):
  def distance(x, y):
    nom = a * x + b * y + c
    if nom == 0:
      pos = 0
    elif (nom < 0 and b < 0) or (nom > 0 and b > 0):
      pos = -1
    else:
      pos = 1
    return (np.absolute(nom) / np.sqrt(a**2 + b**2), pos)
  return distance
# Corrected coordinate tuple syntax
points = [(3, 5), (1, 8), (1.1, 3.9)]
fig, ax = plt.subplots()
ax.set_xlabel("sweetness")
ax.set_ylabel("sourness")
ax.set_xlim([-1, 6])
ax.set_ylim([-1, 8])
X = np.arange(-0.5, 5, 0.1)
size = 10
for index, (x, y) in enumerate(points):
  if index == 0:
    ax.plot(x, y, "o", color="darkorange", markersize=size)
  else:
    ax.plot(x, y, "oy", markersize=size)
step = 0.05
for x_val in np.arange(0, 1 + step, step):
  slope = np.tan(np.arccos(x_val))
```

```
    dist4line1 = create_distance_function(slope, -1, 0)
    Y = slope * X
    results = []
    for point in points:
        results.append(dist4line1(*point))
    if results[0][1] != results[1][1]:
        ax.plot(X, Y, "g-")
    else:
        ax.plot(X, Y, "r-")
plt.show()
```

OUTPUT:



B) Hopfield Network Model for Associative Memory

Aim: To implement a Hopfield network model for associative memory.

Theory: Associative memory stores patterns in the network and retrieves a related stored pattern when a noisy or incomplete version is presented.

Result: The required pattern is stored and can be recalled through associative memory.

PRACTICAL: 8

WRITE-UP – A) Membership Operators: in, not in

Aim: To implement membership operators 'in' and 'not in'.

Theory: Membership operators test whether an element exists in a collection. 'in' returns True when an element is present, while 'not in' returns True when it is absent.

Result: The program determines whether the specified lists contain matching elements.

A) Membership and identity operators in, not in

CODE:

```
def overlapping(list1,list2):
  for i in range(0,len(list1)):
    for j in range(0,len(list2)):
      if list1[i]==list2[j]:
        return True
  return False
n=int(input("Input number of elements for list1:"))
m=int(input("Input number of elements for list2:"))
list1=[]
list2=[]
for i in range(0,n):
  l1=float(input("Enter List1 value"+str(i)+"="))
  list1.append(l1)
for i in range(0,m):
  l2=float(input("Enter List2 value"+str(i)+"="))
  list2.append(l2)
if(overlapping(list1,list2)):
  print("List are Overlapping")
```

```
else:
  print("List are not Overlapping")
```

OUTPUT:

```
Input number of elements for list1:2
Input number of elements for list2:2
Enter List1 value0=2
Enter List1 value1=3
Enter List2 value0=24
Enter List2 value1=4
List are not Overlapping
```

B) Identity Operators: is, is not

Aim: To implement identity operators 'is' and 'is not'.

Theory: Identity operators compare object identity. 'is' checks whether two references point to the same object, while 'is not' checks that they do not.

Result: The identity relationship between the given objects is determined.

PRACTICAL: 9

WRITE-UP – A) Find Ratios using Fuzzy Logic

Aim: To find similarity ratios between strings using fuzzy matching techniques.

Theory: Fuzzy string matching calculates similarity between text values using different comparison measures such as complete, partial and token-based ratios.

Result: Similarity ratios and the best matching string are obtained for the given inputs.

A) **Find ratios using Fuzzy logic**

CODE:

```
from fuzzywuzzy import fuzz, process

s1 = "I Love Fuzzys For Fuzzy"

s2 = "I Am Loving Fuzzys For Fuzzy"

print("FuzzyWuzzy Ratio:",fuzz.ratio(s1.lower(), s2.lower()))

print("FuzzyWuzzy Partial Ratio",fuzz.partial_ratio(s1.lower(), s2.lower()))

print("Fuzz Wuzzy TokenSort Ratio:",

fuzz.token_sort_ratio(s1.lower(), s2.lower()))

print("FuzzWuzzy TokenSet Ratio: ",

fuzz.token_set_ratio(s1.lower(), s2.lower()))

print("fuzzywuzzy WRatio:", fuzz.WRatio(s1.lower(), s2.lower()))

query='Fuzzy For Fuzzy'

choices=['Fuzzy For Fuzzy', 'Fuzzy Fuzzy', 'g. For Fuzzy']

print("List Of Ratios:")

print(process.extract(query, choices),"\n")

print("Best Among The Above List:", process.extractOne(query, choices))
```

OUTPUT:

```
FuzzyWuzzy Ratio: 86
FuzzyWuzzy Partial Ratio 87
Fuzz Wuzzy TokenSort Ratio: 86
FuzzWuzzy TokenSet Ratio:  88
fuzzywuzzy WRatio: 86
List Of Ratios:
[('Fuzzy For Fuzzy', 100), ('Fuzzy Fuzzy', 95), ('g. For Fuzzy', 95)]

Best Among The Above List: ('Fuzzy For Fuzzy', 100)
```

WRITE-UP – B) Tipping Problem using Fuzzy Logic

Aim: To solve the tipping problem using fuzzy logic.

Theory: Fuzzy logic represents food quality and service quality using membership functions. Fuzzy rules are then applied to determine an appropriate tip level.

Result: The fuzzy system produces a tip value based on the quality and service inputs.

B) Solve Tipping problem using fuzzy logic

Source Code:

```
!pip install scikit-fuzzy
import numpy as np
import skfuzzy as fuzz
from skfuzzy import control as ctrl
#New Antecedent/Consequent objects hold universe variables and membership
#functions
quality= ctrl. Antecedent(np.arange(0, 11, 1), 'quality')
service = ctrl. Antecedent(np.arange(0, 11, 1), 'service')
tip = ctrl.Consequent(np.arange(0, 26, 1), 'tip')
#Auto-membership function population is possible with automf(3, 5, or 7)
quality.automf(3)
service.automf(3)
#custom membership functions can be built interactively with a familiar,
#Pythonic API
tip['low'] = fuzz.trimf(tip.universe, [0, 0, 13])
tip['medium'] = fuzz.trimf(tip.universe, [0,13,25])
tip['high'] = fuzz.trimf(tip.universe, [13, 25, 25])
"""
```

To help understand what the membership looks like, use the ''view'' methods.

"""

#you can see how these look with view()

quality['average'].view()

"""

..image:: PLOT2RST.current_figure

"""

service.view()

"""

..image:: PLOT2RST.current_figure

"""

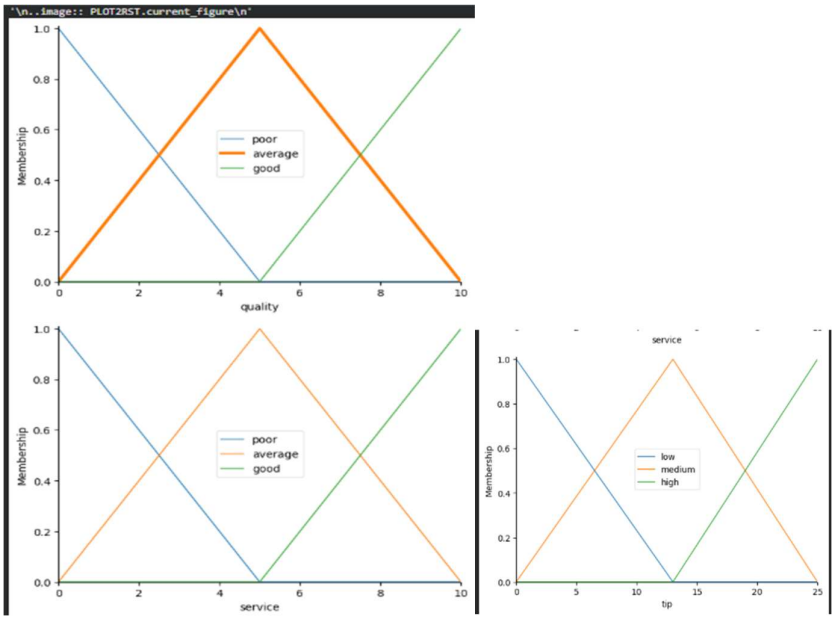
tip.view()

"""

..image:: PLOT2RST.current_figure

"""

OUTPUT:



PRACTICAL 10

WRITE-UP – A) Simple Genetic Algorithm

Aim: To implement a simple genetic algorithm.

Theory: A genetic algorithm searches for a good solution using a population of candidate solutions. It uses fitness evaluation, selection, mating and mutation to produce improved generations.

Result: The population evolves toward the specified target solution.

A) Implement of simple genetic algorithm

CODE:

```
import random
# Number of individuals in each generation
POPULATION_SIZE = 100
# Valid genes
GENES =
"abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ
1234567890,-;:_!#%&/()=?@${[]}"
# Target string to be generated
TARGET = "I love SIES College"
class Individual(object):
  """
  Class representing individual in population
  """
  def __init__(self, chromosome):
    self.chromosome = chromosome
    self.fitness = self.cal_fitness()
  @classmethod
  def mutated_genes(cls):
    """
    Create random genes for mutation
    """
    global GENES
    gene = random.choice(GENES)
    return gene
  @classmethod
  def create_gnome(cls):
    """
```

```
    Create chromosome or string of genes
    """
    global TARGET
    gnome_len = len(TARGET)
    return [cls.mutated_genes() for _ in range(gnome_len)]
def mate(self, par2):
    """
    Perform mating and produce new offspring
    """
    # chromosome for offspring
    child_chromosome = []
    for gp1, gp2 in zip(self.chromosome, par2.chromosome):
        # random probability
        prob = random.random()
        # if prob is less than 0.45, insert gene from parent 1
        if prob < 0.45:
            child_chromosome.append(gp1)
        # if prob is between 0.45 and 0.90, insert gene from parent 2
        elif prob < 0.90:
            child_chromosome.append(gp2)
        # otherwise insert random gene(mutate), for maintaining diversity
        else:
            child_chromosome.append(self.mutated_genes())
    # create new Individual (offspring) using generated chromosome
    return Individual(child_chromosome)
def cal_fitness(self):
    """
    Calculate fitness score, it is the number of
    characters in string which differ from target string.
    """
    global TARGET
    fitness = 0
    for gs, gt in zip(self.chromosome, TARGET):
        if gs != gt:
```

```
            fitness += 1
        return fitness
# Driver code
def main():
    global POPULATION_SIZE
    # current generation
    generation = 1
    found = False
    population = []
    # create initial population
    for _ in range(POPULATION_SIZE):
        gnome = Individual.create_gnome()
        population.append(Individual(gnome))
    while not found:
        # sort the population in increasing order of fitness score
        population = sorted(population, key=lambda x: x.fitness)
        # if the individual having lowest fitness score is 0,
        # then we know that we have reached the target and break the loop
        if population[0].fitness <= 0:
            found = True
            break
        # Otherwise generate new offsprings for new generation
        new_generation = []
        # Perform Elitism, that means 10% of fittest population goes to the next
generation
        s = int((10 * POPULATION_SIZE) / 100)
        new_generation.extend(population[:s])
        # From 50% of fittest population, individuals will mate to produce
offspring
        s = int((90 * POPULATION_SIZE) / 100)
        for _ in range(s):
            parent1 = random.choice(population[:50])
            parent2 = random.choice(population[:50])
            child = parent1.mate(parent2)
```

```
            new_generation.append(child)
        population = new_generation
        print("Generation: {}\tString: {}\tFitness: {}".format(
            generation,
            "".join(population[0].chromosome),
            population[0].fitness
        ))
        generation += 1
    print("Generation: {}\tString: {}\tFitness: {}".format(
        generation,
        "".join(population[0].chromosome),
        population[0].fitness
    ))
if __name__ == "__main__":
    main()
```

OUTPUT:

```
Generation: 150 String: I love SIEj College     Fitness: 1
Generation: 151 String: I love SIEj College     Fitness: 1
Generation: 152 String: I love SIEj College     Fitness: 1
Generation: 153 String: I love SIEj College     Fitness: 1
Generation: 154 String: I love SIEj College     Fitness: 1
Generation: 155 String: I love SIEj College     Fitness: 1
Generation: 156 String: I love SIEj College     Fitness: 1
Generation: 157 String: I love SIEj College     Fitness: 1
Generation: 158 String: I love SIEj College     Fitness: 1
Generation: 159 String: I love SIEj College     Fitness: 1
Generation: 160 String: I love SIEj College     Fitness: 1
Generation: 161 String: I love SIEj College     Fitness: 1
Generation: 162 String: I love SIEj College     Fitness: 1
Generation: 163 String: I love SIEj College     Fitness: 1
Generation: 164 String: I love SIEj College     Fitness: 1
Generation: 165 String: I love SIEj College     Fitness: 1
Generation: 166 String: I love SIEj College     Fitness: 1
Generation: 167 String: I love SIEj College     Fitness: 1
Generation: 168 String: I love SIEj College     Fitness: 1
Generation: 169 String: I love SIEj College     Fitness: 1
Generation: 170 String: I love SIEj College     Fitness: 1
Generation: 171 String: I love SIEj College     Fitness: 1
```

WRITE-UP – B) City and Fitness Classes using Genetic Algorithm

Aim: To create City and Fitness classes for a genetic-algorithm-based solution.

Theory: The City class represents a candidate city or route, while the Fitness class evaluates the quality of that candidate. Genetic operations can then be applied to improve the solution.

Result: Candidate solutions are evaluated using fitness and better solutions can be selected.

B) Create two classes: City and fitness using genetic algorithm

CODE:

```
import numpy
# 1. Parameter initialization
genes = 2
chromosomes = 10
mating_pool_size = 6
offspring_size = chromosomes - mating_pool_size
lb = -5
ub = 5
population_size = (chromosomes, genes)
generations = 4
# 2. Population initialization
population = numpy.random.uniform(lb, ub, population_size)
for generation in range(generations):
  print(f"\n--- Generation: {generation + 1} ---")
  # Calculate fitness (minimizing or maximizing x^2 + y^2; here we show
the sum of squares)
  fitness = numpy.sum(population * population, axis=1)
  print("Current Population:\n", population)
  print("Fitness Values:\n", fitness)
  # 3. Selection: Extract the best parents
  parents = numpy.empty((mating_pool_size, population.shape[1]))
  temp_fitness = fitness.copy()
  for p in range(mating_pool_size):
    # Find index of the fittest chromosome
    fittest_index = numpy.argmax(temp_fitness)
    parents[p, :] = population[fittest_index, :]
    # Set its fitness to negative infinity so it is not selected again in this
generation
    temp_fitness[fittest_index] = -numpy.inf
  print("\nSelected Parents:\n", parents)
  # 4. Crossover (Single-point crossover)
  offspring = numpy.empty((offspring_size, genes))
  for k in range(offspring_size):
    parent1_idx = k % mating_pool_size
```

```
        parent2_idx = (k + 1) % mating_pool_size
        crossover_point = genes // 2
        offspring[k, 0:crossover_point] = parents[parent1_idx,
0:crossover_point]
        offspring[k, crossover_point:] = parents[parent2_idx, crossover_point:]
    # 5. Mutation: Modify a random gene of each offspring
    for index in range(offspring_size):
        random_index = numpy.random.randint(0, genes)
        random_value = numpy.random.uniform(lb, ub, 1)[0]
        offspring[index, random_index] = offspring[index, random_index] +
random_value
    print("\nOffspring after Mutation:\n", offspring)
    # 6. Create new population (Elitism: replace the worst individuals with
offspring)
    population[0:parents.shape[0], :] = parents
    population[parents.shape[0]:, :] = offspring
    print("\nNew Population for next generation:\n", population)
# Final Evaluation
fitness = numpy.sum(population * population, axis=1)
best_index = numpy.argmax(fitness)
best_individual = population[best_index, :]
best_fitness = fitness[best_index]
print("\n==================================")
print("Best Individual:\n", best_individual)
print("Best Individual's Fitness:\n", best_fitness)
```

OUTPUT:

